

Кafka-пайплайн обработки юридических документов

Архитектурное предложение для «Консультант Плюс»

Домашнее задание 8

Проектирование надёжного, масштабируемого
и отказоустойчивого потокового пайплайна

Содержание

1. Разбиение потоков данных на топики	3
1.1. Принципы именования	3
1.2. Таблица топиков	3
1.3. Описание топиков	3
1.4. Единый топик vs. разделённые топики	6
2. Политики хранения	7
2.1. Стратегии очистки	7
2.2. Настройки по топикам	7
2.3. Влияние на дисковое пространство	8
3. Гарантии доставки	9
3.1. Настройки продюсеров	9
3.2. Гарантии доставки по типам данных	9
3.3. Обработка дубликатов и повторных доставок	11
4. Группы потребителей и downstream-сервисы	12
4.1. cg-es-indexer — Elasticsearch-индексатор	12
4.2. cg-graph-builder — Сервис построения графа ссылок	12
4.3. cg-notify-service — Сервис уведомлений	13
4.4. cg-archive-s3 — Архивное хранилище	13
4.5. cg-cache-warm — Прогрев кэша	13
4.6. cg-main-db-updater — Обновление главного хранилища	13
4.7. cg-analytics-spark — Аналитика (Apache Spark Structured Streaming) . .	14
4.8. cg-status-projector — Проектор статусов	14
5. Архитектурная диаграмма	15
6. Заключение	18
6.1. Итоговые решения	18
6.2. Дальнейшие шаги	18

1. Разбиение потоков данных на топики

1.1. Принципы именования

Все топики следуют схеме:

кр.<домен>.<сущность>

где:

- кр — префикс продукта («Консультант Плюс»);
- <домен> — docs, events, meta, notify, refs, search, internal, dlq;
- <сущность> — конкретный тип данных или события.

Имена содержат только строчные буквы, цифры и точки. Это упрощает управление ACL и мониторинг.

1.2. Таблица топики

Топик	Ключ сообщения	Партиций	Объём (год)	Политика
кр.docs.legislative	doc_id	16	150 К событий	compacted + delete, 3 года
кр.docs.court	doc_id	64	3 М событий	delete, 2 года
кр.docs.admin	doc_id	32	500 К событий	delete, 2 года
кр.events.status	doc_id	32	2 М событий	delete, 90 дней
кр.meta.versions	doc_id	16	4 М событий	compacted (вечно)
кр.refs.links	source_doc_id	16	10 М событий	compacted (вечно)
кр.notify.outbox	user_id	32	5 М событий	delete, 7 дней
кр.search.delta	doc_id	32	5 М событий	delete, 3 дня
кр.internal.replication	doc_id	8	служебный	delete, 1 день
кр.dlq	original_topic	8	ошибки	delete, 30 дней

Таблица 1. Топики Kafka и их параметры

1.3. Описание топики

1.3.1. кр.docs.legislative

Содержит полные тела законодательных актов: федеральные законы, кодексы, постановления Правительства, указы Президента. **Каждое сообщение — одна версия документа:** если принята поправка, публикуется новая версия с тем же doc_id (новая редакция).

Ключ: doc_id — гарантирует, что все версии одного закона попадают в одну partition, что важно для compaction и для упорядоченной обработки редакций.

16 партиций: объём 150 К документов/год, пиковая нагрузка 50 сообщений/с при массовых поправках. При одном потоке на партицию достаточно 8, но берём 16 для запаса масштабирования без переразбиения.

Compacted + delete: хотим сохранить **последнюю редакцию** для быстрого доступа (compaction), но при retention.ms~3~года старые редакции всё же вытесняются. Для истории версий используется `kp.meta.versions`.

1.3.2. kp.docs.court

Судебные решения: Конституционный суд, Верховный суд, арбитражные суды всех уровней. Объём значительно выше законодательного — ежегодно публикуется до 3 миллионов документов.

64 партиции: из расчёта 100 документов/с в пиковое время (окончание квартала, массовое рассмотрение дел). Целевой lag — < 10 секунд для ES-индексатора.

Ключ: `doc_id`. Порядок внутри одного дела (апелляция → кассация → надзор) обеспечивается через поле `parent_doc_id` в теле сообщения, а не через Kafka ordering.

1.3.3. kp.docs.admin

Подзаконные акты (приказы ведомств), письма Минфина, ФНС, разъяснения Роструда. Средний объём, но высокая бизнес-значимость для корпоративных подписчиков.

32 партиции: 15 К документов/месяц, равномерная нагрузка; 32 позволяет запустить 32 потока индексатора при необходимости.

1.3.4. kp.events.status

Служебный топик **только для событий смены статуса** документа: { `doc_id`, `event_type`, `effective_date`, `prev_status`, `new_status` }. Примеры: ENTERED_INTO_FORCE, INVALIDATED, AMENDED, REVISION_PUBLISHED.

Отделён от основных топиков намеренно: позволяет сервису уведомлений подписаться **только** на этот топик без разбора полных тел документов. Это снижает нагрузку и упрощает реализацию потребителей.

90-дневный retention: события смены статуса не нужны надолго — потребители либо сразу обрабатывают их, либо после восстановления реплеят за последние 90 дней. Для более длинного горизонта используется `kp.meta.versions`.

1.3.5. `kp.meta.versions`

Compacted-топик, хранящий **последнюю известную версию метаданных** каждого документа: { `doc_id`, `current_version`, `status`, `title`, `doc_type`, `effective_date`, `related_docs[]` }. Аналог материализованного представления.

Только compaction, без delete (`cleanup.policy=compact`): этот топик — единственный источник правды о текущем состоянии документа. Потребитель, запускающийся впервые (`offset=earliest`), получает полный snapshot текущего состояния за одно чтение.

16 партиций: нагрузка меньше, чем на `docs`-топики, так как сообщение пишется только при обновлении метаданных; важна compaction-эффективность.

1.3.6. `kp.refs.links`

Compacted-топик рёбер графа ссылок: { `source_doc_id` → [{ `target_doc_id`, `ref_type`, `anchor` }] }. Сервис построения графа пишет сюда обновлённый список исходящих ссылок при каждом парсинге нового тела документа.

Compaction по `source_doc_id`: хранится только актуальный список исходящих ссылок — так как документ не меняет базовую структуру ссылок, старые версии списка не нужны.

1.3.7. `kp.notify.outbox`

Очередь уведомлений для конечных пользователей: { `user_id`, `subscription_type`, `doc_id`, `event_summary`, `delivery_channels[]` }. Пишется сервисом уведомлений на основе подписок пользователей.

Ключ `user_id`: гарантирует упорядоченную доставку уведомлений одному пользователю; позволяет email/push-воркерам равномерно распределить нагрузку.

7 дней: уведомления теряют ценность быстро; неотправленные более 7 дней уведомления можно считать просроченными.

1.3.8. `kp.search.delta`

Дельта-топик для Elasticsearch: содержит минимальный набор полей, необходимых для переиндексации (`doc_id`, `title`, `body_text`, `doc_type`, `status`, `tags`). Пишется фильтром после `kp.docs.*`.

3 дня: ES-индексатор должен обрабатывать дельты с лагом не более нескольких минут; 3 дня — буфер на случай полной недоступности кластера ES.

1.3.9. kp.dlq

Dead Letter Queue для всех потребителей. Сообщение попадает сюда, если потребитель не смог обработать его после N попыток. Ключ — имя исходного топика + исходный ключ.

1.3.10. kp.internal.replication

Служебный топик для внутренней репликации метаданных между дата-центрами через MirrorMaker 2. Не читается бизнес-потребителями.

1.4. Единый топик vs. разделённые топики

Критерий	Единый топик <code>kp.docs.all</code>	Разделённые топики
Управление retention	Единая политика для всех типов — неудобно (суд. решения нужны дольше законов)	Каждый топик имеет свой retention
Нагрузка на потребителей	Каждый потребитель читает всё, фильтруя по типу — расточительно	Потребитель подписывается только на нужный домен
Масштабирование	Один горячий топик, трудно добавить партиции без остановки	Разные topics масштабируются независимо
Простота мониторинга	Сложно различать lag по типу документа	Отдельный lag per topic
Compaction	Невозможно применить к части сообщений	<code>kp.meta.versions</code> compacted, <code>kp.docs.court</code> delete
Вывод	Подходит только для прототипа	Выбранный подход

Таблица 2. Сравнение единого и разделённых топиков

Вывод: **разделённые топики** обязательны для production-системы данного масштаба. Единый топик рассматривался как упрощение архитектуры, но проигрывает по всем ключевым параметрам.

2. Политики хранения

2.1. Стратегии очистки

Кafka поддерживает две стратегии очистки сегментов (`log.cleanup.policy`):

- **delete** — удаление сегментов по истечении `retention.ms` или при превышении `retention.bytes`. Подходит для событийных данных (статусы, уведомления).
- **compact** — удаление старых версий сообщений с одним ключом, сохранение только последней. Подходит для «снимков состояния» (метаданные документа, граф ссылок).
- **compact,delete** — комбинация: сначала compaction удаляет устаревшие версии ключа, затем delete вытесняет старые сегменты. Используется для `kp.docs.legislative`, где нужны и последняя редакция, и временное окно.

2.2. Настройки по топикам

Топик	cleanup.policy	retention.ms	retention.bytes	Обоснование
kp.docs.legislative	compact,delete	94 608 000 000 (3 года)	10 GB/partition	Законы должны быть доступны 3 года для истории редакций; compaction сохраняет последнюю версию бессрочно
kp.docs.court	delete	63 072 000 000 (2 года)	50 GB/partition	Суд. решения — большой объём; 2 года покрывают апелляционные циклы; старые уходят в S3-архив
kp.docs.admin	delete	63 072 000 000 (2 года)	20 GB/partition	Аналогично court, срок действия большинства актов — до 5 лет, но в Kafka держим 2 года
kp.events.status	delete	7 776 000 000 (90 дней)	5 GB/partition	Статусные события — краткосрочные триггеры; 90 дней — достаточно для повторной обработки после сбоя
kp.meta.versions	compact	−1 (infinity)	2 GB/partition	Единственный источник актуального состояния; хранится вечно; compaction минимизирует объём
kp.refs.links	compact	−1 (infinity)	3 GB/partition	Граф ссылок — персистентный; хранится по-

				следнее известное состояние ссылок
kp.notify.outbox	delete	604 800 000 (7 дней)	1 GB/partition	Уведомления теряют ценность через несколько часов; 7 дней — safety buffer
kp.search.delta	delete	259 200 000 (3 дня)	5 GB/partition	ES-индексатор должен обрабатывать дельты в реальном времени; 3 дня на случай outage
kp.dlq	delete	2 592 000 000 (30 дней)	2 GB/partition	30 дней для ручного разбора и переработки ошибок
kp.internal.replication	delete	86 400 000 (1 день)	500 MB/partition	Служебный топик, данные реплицируются почти сразу

Таблица 3. Политики хранения по топикам

2.3. Влияние на дисковое пространство

Оценка суммарного объёма на одном брокере (при RF = 3 данные хранятся на каждом из 3 брокеров):

Топик	Партиций	Max GB/partition	Max GB (топик)
kp.docs.legislative	16	10	160
kp.docs.court	64	50	3 200
kp.docs.admin	32	20	640
kp.events.status	32	5	160
kp.meta.versions	16	2	32
kp.refs.links	16	3	48
kp.notify.outbox	32	1	32
kp.search.delta	32	5	160
kp.dlq	8	2	16
Итого (1 брокер)			4 450 GB
Итого (3 брокера)			13 350 GB

Таблица 4. Оценка дискового пространства

Доминирующий топик — `kp.docs.court` из-за большого объёма судебных решений. Рекомендуется выделить брокерам диски минимум **6 TB SSD** каждому с запасом 2×. Для `kp.docs.court` можно рассмотреть **tiered storage** (разгрузка холодных сегментов в S3), что снизит требования к локальному диску до 2 TB на брокер.

3. Гарантии доставки

3.1. Настройки продюсеров

Все продюсеры используют единый базовый профиль:

```
# Надёжность
acks=all                                # ждём подтверждения от всех ISR
enable.idempotence=true                 # идемпотентный продюсер (sequence
number)
max.in.flight.requests.per.connection=5 # допустимо при idempotence=true
(Kafka >= 1.1)
retries=2147483647                      # бесконечные повторы до
delivery.timeout.ms
delivery.timeout.ms=120000              # 2 мин — максимальное время одной
попытки

# Производительность
linger.ms=5                             # небольшая задержка для батчинга
batch.size=65536                         # 64 KB батч
compression.type=lz4                     # LZ4: быстрее zstd на запись, хорошее
сжатие
buffer.memory=67108864                  # 64 MB in-memory буфер продюсера
```

acks=all + min.insync.replicas=2: сообщение считается записанным только после подтверждения 2 из 3 реплик. Брокер, упавший в момент записи, не приведёт к потере данных.

enable.idempotence=true: Kafka гарантирует exactly-once на уровне продюсера внутри одной сессии. При retry продюсер не создаёт дубликаты благодаря producer ID + sequence number.

max.in.flight.requests.per.connection=5: с idempotence = true Kafka (начиная с версии 1.1) корректно переупорядочивает до 5 in-flight-запросов, что даёт хорошую пропускную способность без риска нарушения порядка.

3.2. Гарантии доставки по типам данных

3.2.1. At-least-once (основные потоки)

Используется для топиков `kp.docs.*`, `kp.events.status`, `kp.notify.outbox`, `kp.search.delta`. Достаточно, потому что потребители **идемпотентны** — повторная обработка одного документа не нарушает корректность.

Механизм идемпотентности потребителей:

Потребитель	Ключ идемпотентности	Реализация
ES-индексатор	doc_id + version_hash	Upsert по _id = doc_id, ES перезаписывает документ с тем же ID — безопасно
Архив S3	s3://kp-archive/{doc_type}/{doc_id}/{version}	S3 Put idempotent: повторная запись одного пути — перезапись файла без побочных эффектов
Сервис уведомлений	notification_id = hash(user_id+doc_id+event_type+date)	Уведомление с таким ID уже в БД — skip; иначе — отправка
Граф ссылок	source_doc_id + version	UPSERT рёбер по (source, target, version); дубликат — no-op

Таблица 5. Идемпотентность downstream-потребителей

Настройки потребителей для at-least-once:

```
enable.auto.commit=false           # ручной коммит offset после успешной
обработки
auto.offset.reset=earliest         # при новом group.id читать с начала
max.poll.records=500               # батч на один poll() — не перегружаем
обработчик
max.poll.interval.ms=300000        # 5 мин — максимум между poll() (для
тяжёлых ES-батчей)
session.timeout.ms=45000           # 45 с — таймаут heartbeat до rebalance
```

3.2.2. Exactly-once (критичные операции)

Применяется **только** для обновления главного хранилища документов (kp-main-db-updater) — сервиса, который записывает данные из kp.docs.* в основную реляционную БД. Потеря или дублирование записи здесь создаёт юридически значимые ошибки.

Реализация через Kafka Transactions:

```
// Продюсер с транзакционным ID
Properties props = new Properties();
props.put("transactional.id", "kp-main-db-updater-" + partitionId);
props.put("enable.idempotence", "true");

KafkaProducer<String, Document> producer = new KafkaProducer<>(props);
producer.initTransactions();

// В цикле обработки:
ConsumerRecords<String, Document> records =
consumer.poll(Duration.ofMillis(500));
producer.beginTransaction();
try {
    for (ConsumerRecord<String, Document> record : records) {
```

```

        mainDb.upsert(record.value()); // запись в БД
        producer.send(new ProducerRecord<>( // подтверждение в audit-
ТОПИК
            "kp.internal.replication", record.key(), record.value()));
    }
    producer.sendOffsetsToTransaction( // атомарно: коммитим
offset
        currentOffsets(consumer), consumer.groupMetadata());
    producer.commitTransaction();
} catch (Exception e) {
    producer.abortTransaction();
    // consumer не коммитит offset → повторная обработка
    throw e;
}

```

Важно: транзакции Kafka гарантируют exactly-once **внутри Kafka**. Запись в внешнюю БД (PostgreSQL) атомарна только при использовании двухфазного коммита или при идиempотентном upsert по (doc_id, version). В данной архитектуре используется **idempotent upsert** в PostgreSQL: `INSERT ... ON CONFLICT (doc_id) DO UPDATE.`

3.3. Обработка дубликатов и повторных доставок

Retry-стратегия для всех потребителей:

Попытка	Задержка	Действие при исчерпании
1-3	немедленно	Повтор в рамках текущего poll-цикла
4-6	экспоненциальный backoff (1 s → 8 s)	Retry с задержкой
7+	-	Сообщение → kp.dlq; offset коммитится

Таблица 6. Стратегия повторных попыток

Сообщения в kp.dlq содержат оригинальный топик, partition, offset, заголовок с описанием ошибки и тело оригинального сообщения. Команда SRE обрабатывает DLQ через отдельный сервис (cg-dlq-reprocessor) с ручным подтверждением после исправления.

4. Группы потребителей и downstream-сервисы

group.id	Топики	Offset reset	Гарантия
cg-es-indexer	kp.search.delta	earliest	at-least-once
cg-graph-builder	kp.docs.legislative, kp.docs.court, kp.docs.admin	earliest	at-least-once
cg-notify-service	kp.events.status, kp.meta.versions	latest	at-least-once
cg-archive-s3	kp.docs.legislative, kp.docs.court, kp.docs.admin	earliest	at-least-once
cg-cache-warm	kp.meta.versions	earliest	at-least-once
cg-main-db-updater	kp.docs.legislative, kp.docs.admin	earliest	exactly-once
cg-analytics-spark	kp.docs.*, kp.events.status	earliest	at-least-once
cg-dlq-reprocessor	kp.dlq	earliest	at-least-once
cg-status-projector	kp.events.status	earliest	at-least-once

Таблица 7. Consumer groups и их характеристики

4.1. cg-es-indexer — Elasticsearch-индексатор

Назначение: полнотекстовый поиск по документам. Читает `kp.search.delta`, формирует ES bulk-запросы, индексирует документы в индексы `kp-legislative-*`, `kp-court-*`, `kp-admin-*`.

Обработка ошибок: при ошибке от ES (5xx) — exponential backoff (max 5 попыток); при 4xx (некорректный документ) — в DLQ. Lag-алерт при > 10 000 сообщений (> 30 секунд отставания).

Масштабирование: горизонтальное — число экземпляров = число партиций `kp.search.delta` (32). Координация через Kafka Consumer Group API (встроенный rebalance).

Offset-стратегия: `earliest` при первом запуске (полная переиндексация). В штатном режиме — `commit` после успешного bulk-запроса к ES.

4.2. cg-graph-builder — Сервис построения графа ссылок

Назначение: парсинг тела документа, извлечение ссылок на другие документы, запись рёбер графа в `kp.refs.links` и в graph-БД (Neo4j / Apache AGE).

Особенность: этот потребитель сам является продюсером (пишет в `kp.refs.links`). Использует idempotent producer; при повторной обработке документа перезаписывает список исходящих ссылок.

Lag-допустимость: высокая — граф ссылок не критичен в режиме реального времени; допустимый lag — до 10 минут.

4.3. cg-notify-service — Сервис уведомлений

Назначение: формирование персонализированных уведомлений пользователям о новых редакциях, вступлении в силу, утрате силы — на основе их подписок.

Логика: читает `kp.events.status` → проверяет, есть ли у пользователей подписки на данный `doc_id` → пишет уведомления в `kp.notify.outbox` → отдельный воркер (`cg-notify-delivery`) отправляет email/push.

Offset-стратегия: `latest` — уведомления о событиях, пропущенных при downtime, не нужны (пользователь обнаружит изменение при следующем входе через `kp.meta.versions`).

4.4. cg-archive-s3 — Архивное хранилище

Назначение: долгосрочное хранение документов в S3 (Yandex Object Storage / AWS S3). Используется для хранения документов старше retention Kafka.

Формат: каждый документ → отдельный объект `s3://kp-archive/{doc_type}/{year}/{month}/{doc_id}/{version}.json.gz`. Partitioned Parquet для аналитики: `s3://kp-analytics/{doc_type}/year={Y}/month={M}/part-{N}.parquet`.

Lag-допустимость: очень высокая; допустимый lag — часы. Не влияет на онлайн-сервисы.

4.5. cg-cache-warm — Прогрев кэша

Назначение: поддержание Redis-кэша актуальными метаданными документов для быстрого доступа (1 мс vs 50 мс из БД). Читает `kp.meta.versions` (compacted), поэтому при старте получает полный актуальный снимок.

Ключи Redis: `meta:{doc_id}` → TTL без истечения (обновляется при каждом новом сообщении в топике).

4.6. cg-main-db-updater — Обновление главного хранилища

Назначение: единственный потребитель с **exactly-once** гарантией. Пишет законодательные и подзаконные акты в основное PostgreSQL-хранилище.

Параллелизм: один инстанс на партицию (16 для `kp.docs.legislative`, 32 для `kp.docs.admin`). Транзакционный ID содержит номер партиции.

4.7. cg-analytics-spark — Аналитика (Apache Spark Structured Streaming)

Назначение: построение аналитических агрегатов: тренды по типам документов, частота изменений, ссылочная активность. Результаты → Apache Iceberg / ClickHouse.

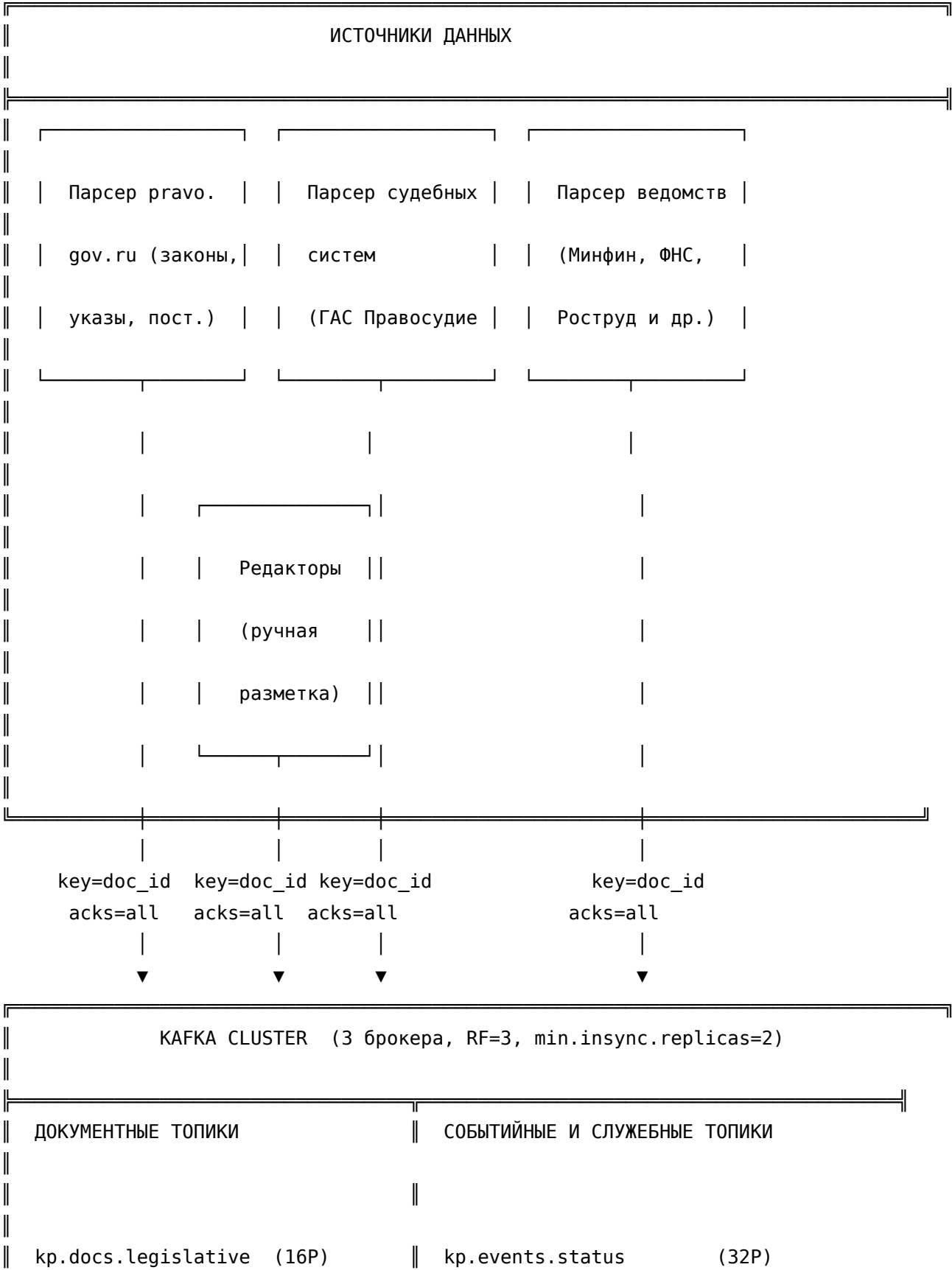
Offset-стратегия: `earliest` с checkpointing в HDFS/S3. Задержка processing — acceptable up to 5 minutes (micro-batch).

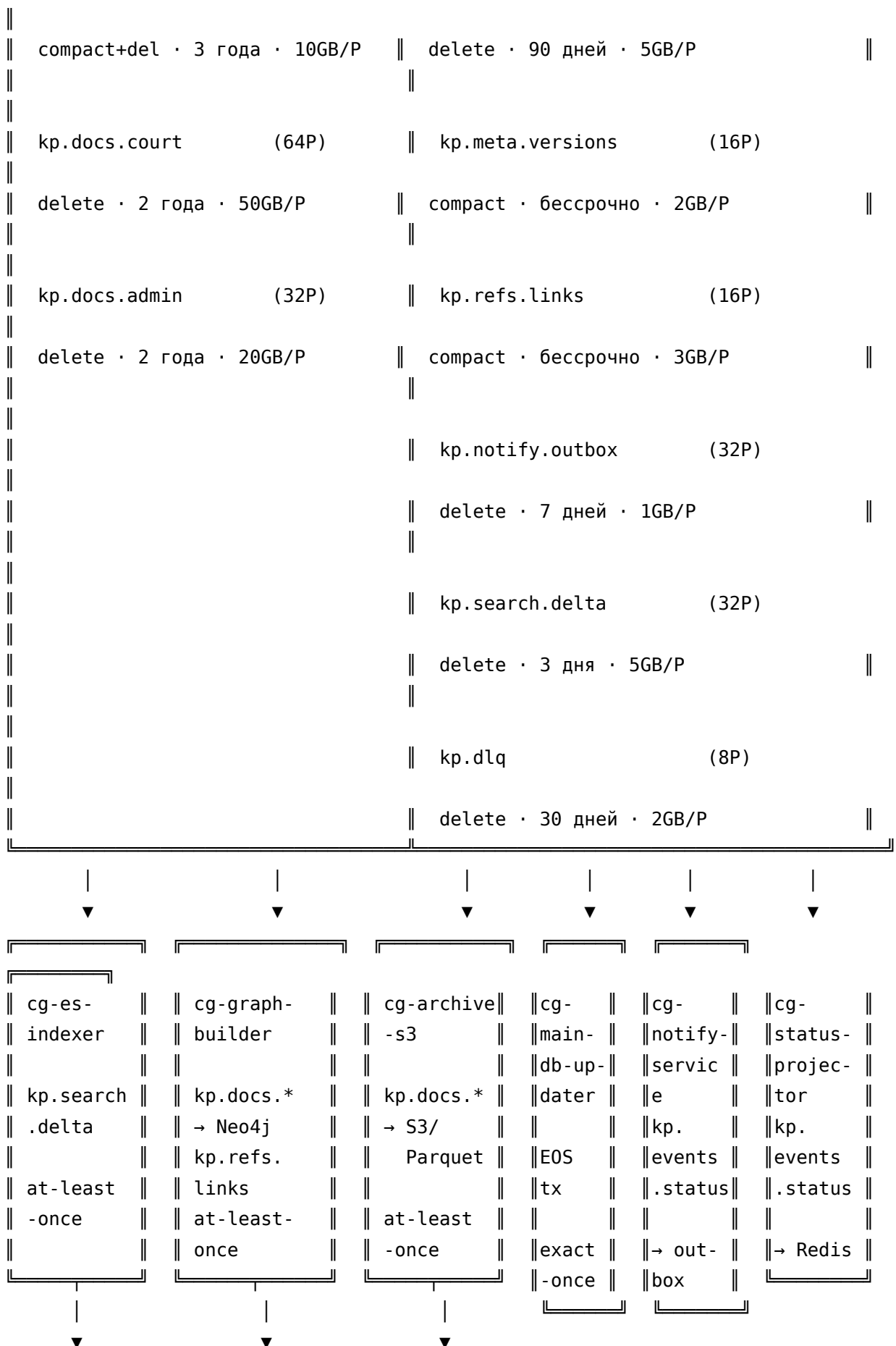
4.8. cg-status-projector — Проектор статусов

Назначение: поддержание проекции текущих статусов всех документов в Redis (второй слой, дополняющий `kr.meta.versions`). Быстрый lookup: «является ли документ X действующим прямо сейчас?».

Реализация: Redis Hash `status:{doc_id}` → { `status`, `effective_date`, `invalidated_date` }.

5. Архитектурная диаграмма





Elasticsearch	Neo4j / AGE	Yandex Object Store
(Full-text	(граф ссылок)	+ Apache Iceberg
search)		(аналитика)

cg-cache-warm

kp.meta.versions →

Redis (мета-кэш)

cg-analytics-spark

kp.docs.* +

kp.events.status →

ClickHouse / Iceberg

cg-dlq-reprocessor

kp.dlq →

ручной разбор

6. Заключение

6.1. Итоговые решения

Аспект	Принятое решение
Топология топигов	10 топигов с разделением по домену и типу данных; отказ от единого топики в пользу специализированных
Именованье	кр.<домен>.<сущность> — предсказуемо, легко управляется ACL и Kafka Schema Registry
Ключевая партиция	doc_id для документных топигов — порядок редакций, эффективный compaction; user_id для notify — равномерная нагрузка по пользователям
Retention	Compaction для состояний (meta.versions, refs.links); delete с разными горизонтами для событийных потоков; tiered storage рекомендован для kp.docs.court
Гарантии доставки	At-least-once + идемпотентные потребители для 8 из 9 групп; exactly-once транзакции только для cg-main-db-updater
Продюсеры	acks=all, idempotence=true, max.in.flight=5, LZ4-компрессия
Репликация	RF=3, min.insync.replicas=2 — переживает выход из строя одного брокера без потери доступности записи
Обработка ошибок	DLQ-топик + exponential backoff (до 6 попыток) → manual review

Таблица 8. Сводная таблица архитектурных решений

6.2. Дальнейшие шаги

1. **Schema Registry** (Confluent / Apicurio): все сообщения в Avro/Protobuf со схемами; обеспечивает эволюцию схем без поломки потребителей.
2. **Kafka Connect**: вместо самописных парсеров использовать коннекторы для pravo.gov.ru (HTTP source connector) и для PostgreSQL (Debezium CDC).
3. **Tiered Storage**: для kp.docs.court подключить Kafka Tiered Storage (GA с Kafka 3.6) или Apache Pinot для аналитики по cold data.
4. **Kafka Streams**: реализовать cg-status-projector и cg-graph-builder как Kafka Streams-приложения — это упростит управление состоянием и checkpoint.
5. **Мониторинг**: Prometheus + Grafana с алертами на kafka.consumer.lag > 50000, under-replicated partitions > 0, producer request rate drop > 50%.
6. **MirrorMaker 2**: активная-пассивная репликация в резервный дата-центр через kp.internal.replication; RTO < 15 минут при сбое основного кластера.